QCon
全球软件开发大会

InfoQ 技术大会

从算子调优到推理自治

构建 MaaS 场景下的 AI Inference 自动优化闭环

杨林枫
阿里巴巴 TRE - AI 系统工程



分享路线与听众收获

从真实工况出发，理解一次算子优化怎样成为可验证的服务收益。

01 哪些优化工作可以自动化完成

哪些高度依赖专家的工作可以交给系统?

02 部署配置探索与优化任务生成

如何选择部署方案，再从运行 Trace 中生成算子任务?

03 Kernel Agent 如何持续优化一个算子

如何组织实验、利用反馈，并确认收益真实?

04 如何将算子收益兑现成模型收益

Kernel 变快后，怎样接回框架、验证服务指标?

05 如何开始自己的实践

最小需要准备什么，怎样逐步扩展到推理自治?

LLM 推理算子优化的人力需求，来自于运行条件的多样性

目前模型算法逐步收敛，但庞大用户基数所带来的多样的推理需求仍带来了多样的优化需求



长短上下文混合
并发与突发流量
缓存命中与多模态

请求工况



集中式、E/P/D 分离
并行与凑批策略
引擎和量化配置

部署形态



算力、带宽与显存
卡间互连和通信
编译器与 Kernel 栈

硬件平台



首 token 与生成延迟
吞吐和资源利用率
容量约束与服务成本

服务目标

传统调优：专家贯穿全程，重复工作反复发生

AS-IS　每次模型、卡型或工况变化，都要重新走一遍。



专家判断跨越模型、引擎和硬件，复现、搬运信息与重复验证又消耗大量时间。

我们的方案：把专家调优经验固化进可持续运行的系统

TO-BE　给定模型、卡型与工况，通过自动化流程持续优化推理服务。



把专家经验固化为知识与工具，用独立评测推动迭代，以模型收益验收。

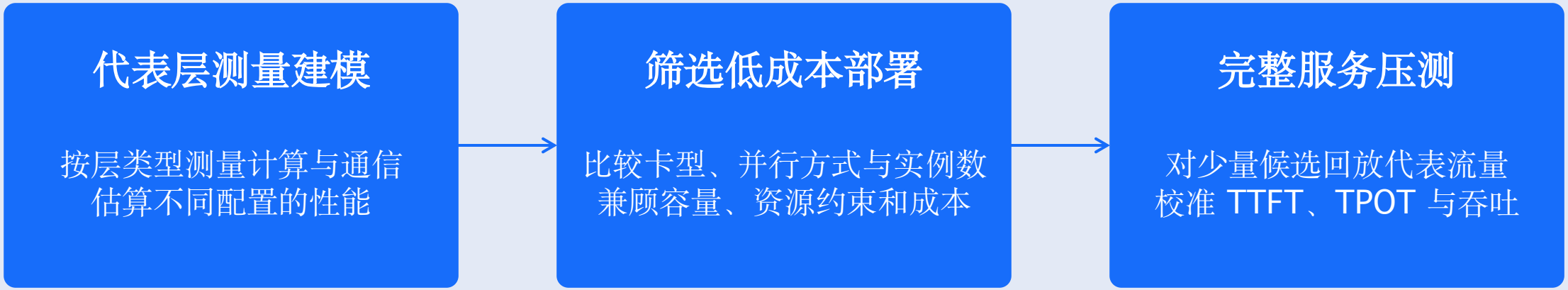
02

部署配置探索与优化任务生成

部署方式决定算子工况与热点。
先筛选部署方案，再从运行 Trace 中生成标准 Benchmark。

部署配置探索：用性能模型筛选低成本方案

我们正在探索：给定模型、典型流量与时延要求，自动选择部署配置。



目标：在满足服务要求的前提下，降低整套部署的资源成本。

确定部署后，采集运行 Trace，进入框架与算子优化。

框架优化：基于 Timeline分析的迭代闭环

先优化框架执行路径，再从更新后的 Trace 中生成算子任务。



评测通过才更新基线；未通过则回退，保留实验记录。

最终源码与 Timeline 一起交给后续的算子任务生成。

自动算子优化为什么需要标准化 Benchmark

Benchmark 是 Kernel Agent 每轮迭代可重复调用的标准入口。

迭代需要	Benchmark 固定什么	如何支撑优化 Loop
统一入口	可重放的算子优化任务，定义 任务加载、候选调用 与评测接口	每轮从同一入口运行与评价 停止后重载任务 结合代码与检查点继续优化
固定工况	输入、Shape 集合 与目标硬件	代表本次优化的目标工况 多轮迭代始终比较同一道题
评分反馈	硬件利用率、算子平均耗时等 指标定义与汇总口径	评估接近硬件上限的程度 比较工况集的整体收益

固定任务与评分口径，让优化可以反复迭代，也能停止后继续。

如何将生产数据转换成算子Benchmark

Converter Agent 按生产证据构造标准任务，流程如下：



输出带有固定工况、统一运行入口与评分规则的Benchmark，交给 Kernel Agent 迭代。

如何防止 Reward Hacking

Reward Hacking 是 Agent 利用评测漏洞“刷高分”，却没有真正改善算子性能。防范它，是为了确保算子性能的提升是来自正确、完整计算下的真实收益。

关键防线	具体做法	防止的投机方式
评测权限隔离	仅修改候选 Kernel 保护参考、输入与评分规则	修改“裁判” 让分数虚高
开发验收分离	公开合法输入域供开发 隐藏真实 case 做完整验收	只针对少数 Shape 优化
输入不退化	输入的Scale值动态、乱序页表 非零历史状态与续算	省略必要计算 仍碰巧通过测试
完整语义检查	检查输出、状态与异常写入 输入规则变化后重新评测	算子输出正确但状态写错（比如 Linear Attention的 State）

当然，防止Reward Hacking是Harness和自进化中一个非常重要的课题，我们也还在持续探索

03

Kernel Agent 如何持续优化

让 Agent 提出方案，用工具反馈与独立评测推动多轮优化。

Atrex Kernel Agent （AKA）：面向真实工况的自动算子优化

Atrex Kernel Agent（AKA）是我们孵化的自动算子优化工具（github.com/alibaba/atrex-kernel-agent）。

为什么需要 AKA

高性能依赖专家知识

算法、流水与硬件特性共同决定上限

优化依赖多轮实验

单次代码生成难以完成持续的性能搜索

结果需要独立验证

正确性、性能测量与实验进度都要管理

AKA 提供的能力

将知识与设计方法交给 Agent

结合硬件经验提出并实现优化方案

用工具反馈推动迭代

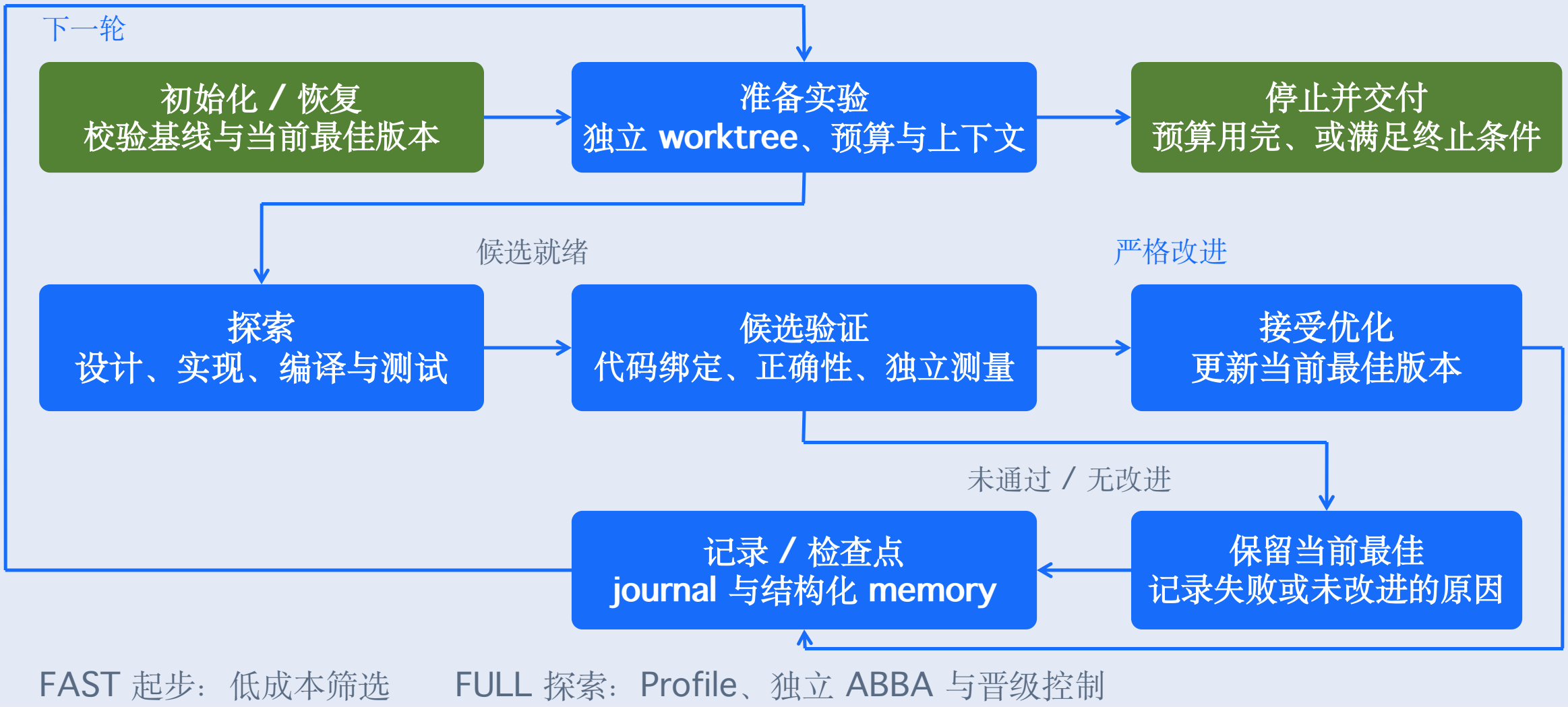
依据 Profile 与实验结果调整方案

用外围 Loop 管理实验

控制预算、独立验证、择优与状态恢复

以 Benchmark 为输入，交付经过验证的 Kernel 和可复现实验记录。

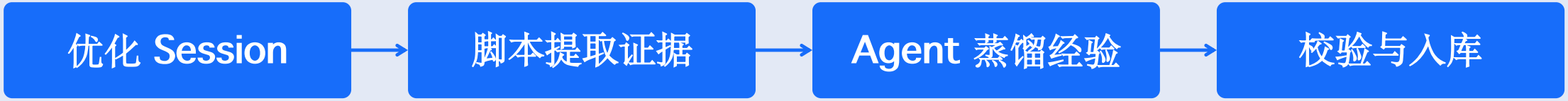
AKA 整体流程：在Harness Loop 约束下的算子自动优化



FAST 起步：低成本筛选　　FULL 探索：Profile、独立 ABBA 与晋级控制

GPU Wiki：从优化 Session 构建可复用知识

构建 Skill 分工：脚本取证、Agent 蒸馏，校验门禁控制知识准入。



代码改动、工具输出
测量结果与尝试记录
成功与失败都保留

对齐改动与测量
区分实测与自述
形成可追溯证据包

提炼机制与适用条件
保留有效策略与反例
形成结构化知识候选

核对数字与原始代码
去重脱敏、冲突复核
经 Wiki Gate 准入

如何供下一轮优化使用

Kernel Wiki 按架构、算子与症状检索，去重并控制返回量。
结合 Hardware Wiki 的硬件事实形成 Plan，记录实际采纳的知识引用。

新 Session 继续沉淀经验，知识带着证据和适用条件进入下一轮优化。

细粒度 Profile：把算子内部“哪里慢”定位到指令级别

反馈工具	观察内容	对设计的帮助
NCU / 硬件指标	吞吐、带宽、occupancy、stall	判断主要资源约束
IKeT / 内核时间线	warp 角色、TMA / MMA、barrier	定位具体同步和流水等待
Micro-benchmark	单独测试某个机制或局部改动	验证瓶颈假设是否成立

硬件指标缩小范围，内核事件解释等待，局部实验再验证机制。

例如 GDN：scalar-TMA warp 与流水阶段的调整，需要内部时间线提供证据。

Plan 设计：把流水设计放在指令调优之前

专家通常自顶向下考虑：先算法，再流水，最后做微架构调优。

算法维度	流水维度	微架构维度
等价数学变换 例如 online softmax	组织数据搬运、计算与同步 设计分工和阶段重叠	指令选择与调度 优化局部实现

我们观察到，Agent 容易直接从指令层入手，过早限制了优化收益空间。

算法级创新目前仍难稳定自动化，Plan 先从流水设计切入。

流水方案与资源约束

tile / layout、warp 分工与流水级数
核算寄存器、片上存储及并发度

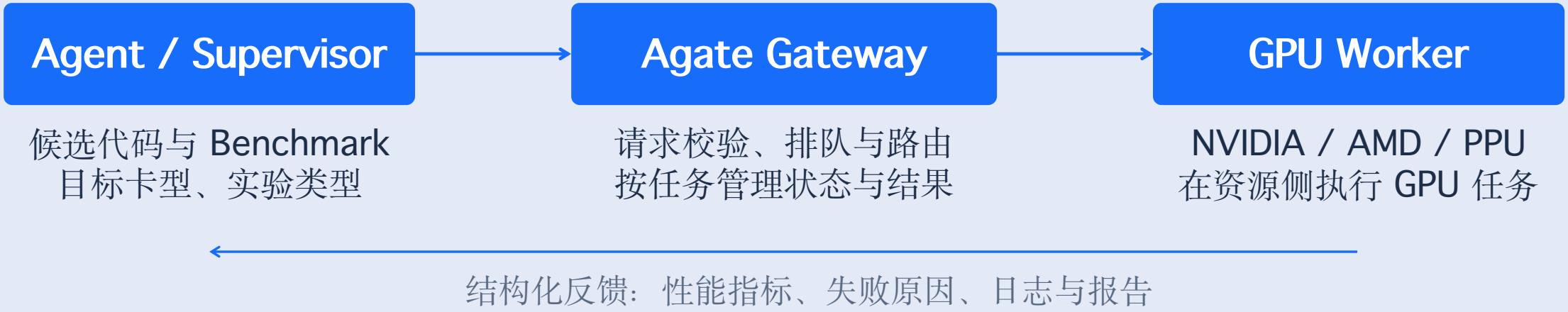
micro-benchmark 校准静态估算

用实测吞吐与延迟估算各阶段耗时
判断计算、访存与同步能否有效重叠

Plan 输出流水结构、资源预算和性能预期，再交给 Agent 编码与实测验证。

Atrex Gateway (Agate)：Agent 的统一 GPU 实验入口

把找卡、配环境和运行工具交给网关，让 Agent 专注优化实验。



统一工具与异构资源

编译、评测、分层 Profile 与反汇编
统一接入 atrex-bench / SOL-ExecBench

一致的评测口径

固定评测器版本，正确性通过后计时
同卡 ABBA 对比，汇总完整 workload

受控执行环境

Worker 预置工具链，任务使用独立目录
依赖按任务管理，隔离强度由后端保证

可恢复的实验任务

同一任务 ID 跟踪状态，断线后继续查询
区分代码与环境故障，避免盲目重跑

公开榜单验证了 AKA 的算子优化能力

No.1

NVIDIA SOL-ExecBench
FlashInfer 算子优化榜

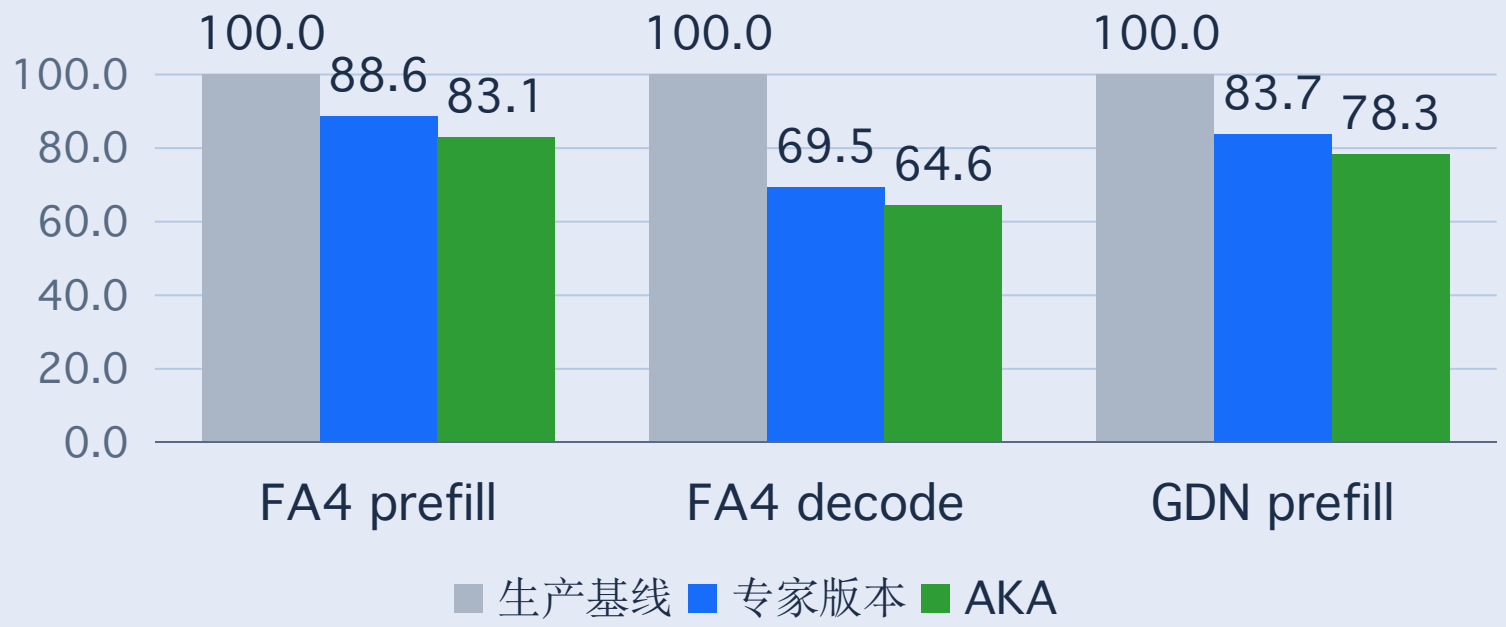
曾支撑 Qwen3.8-Max 获得榜首

接下来要回答：真实模型的输入和接口约束下，还能否保持优势？

Qwen 旗舰模型：三个关键算子整体优于专家

Qwen3.7-Max / Qwen3.8-Max 真实工况，覆盖 60 个 shape。

生产基线时延 = 100，越低越好



较专家加速

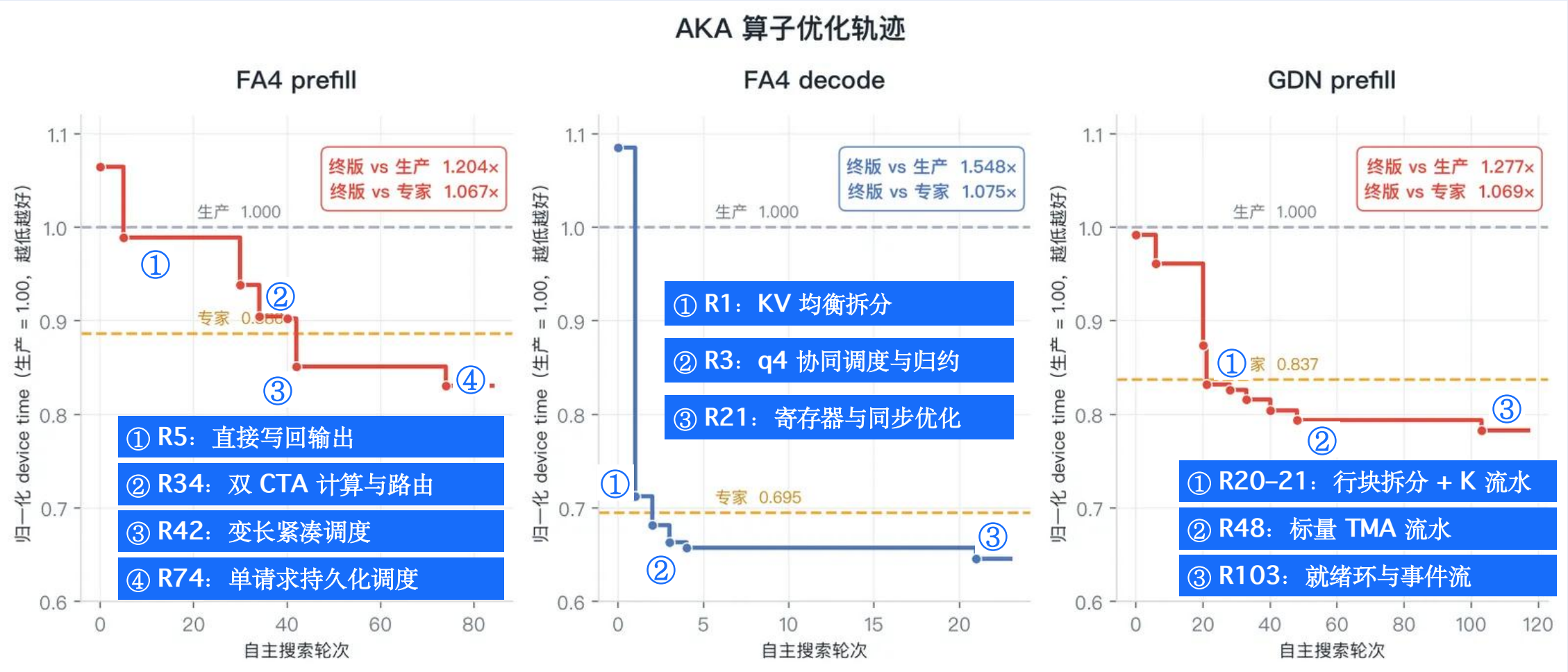
1.067×
1.075×
1.069×

分别覆盖
30 / 20 / 10 shapes

完整 shape 集合的几何平均时延。优于专家的 shape：25/30、12/20、10/10。
FA4 prefill 对比闭源 FlashInfer-TRTLLM，decode / GDN 对比公开 FlashInfer。

真实迭代轨迹：结构性改动带来阶段性提升

R 为搜索轮次，蓝色编号对应实际改动。橙色虚线为专家，时延越低越好。



04

如何把算子收益兑现到模型

接回推理框架，在相同服务工况下验证精度与端到端性能。

模型端到端验证，决定局部优化能否兑现

验证层次	需要回答的问题
算子回归	全 shape 正确性是否保持，收益是否可复测
框架集成	真实调用、状态与后续迭代是否正常
模型精度	轻量检查先挡明显错误，正式结果仍需完整评测
端到端性能	相同部署和流量下，TTFT、TPOT / ITL、吞吐是否改善

NVIDIA 实践：六类算子接入后的模型时延收益

GLM-4.7-Flash，TP=1。25 种组合筛选后，六类算子接回 vLLM。

单算子优化

离线评测加速比，越高越好

接入算子	使用位置	离线加速比
prefill_flash_attention	Prefill	1.30×
fp8_fused_experts	Prefill	1.32×
cutlass_scaled_mm	Decode	1.18×
grouped_topk	路由	3.29×
silu_and_mul	激活	1.58×
greedy_argmax	采样	1.54×

六类算子均通过离线正确性与质量门禁。
离线加速比不等同于模型端到端加速比。

端到端收益（代表工况）

TTFT：ms，TPOT：ms/token

指标 / 工况	优化前	优化后	下降
TTFT 约 1K	66.303	52.301	21.1%
TTFT 约 2K	118.385	82.393	30.4%
TTFT 4K 无缓存	262.026	177.984	32.1%
TPOT 并发 1	8.084	6.432	20.4%
TPOT 并发 8	11.405	10.113	11.3%

完整工况：TTFT ↓8.3%–32.1%
TPOT ↓11.3%–20.4%

模型收益取两轮独立压测均值，第二轮交换 GPU。对照为本次原生服务，仅代表当前模型与工况。

NVIDIA 实践2：GDN 接入后的模型时延收益

Qwen3.6-Air ， NVFP4，TP=2，C=1。端到端结果来自 自动产出的CUDA Gated-Delte-Net 算子接入。

单算子优化（代表候选）

离线加速比，对比 production

候选算子	Shape 数	离线加速比
GDN input projection	12	2.472×
Shared expert MLP	12	2.373×
GDN chunk	11	2.271×
FlashAttention	12	2.116×
GDN gating	12	2.001×
Causal conv1d update	1	1.930×

共 11 个正收益候选，108 个 shape-point
等权 GM 加速 1.635×，不代表模型收益。

端到端收益（GDN 接入）

TTFT：ms，TPOT：ms/token

指标 / 输入长度	优化前	优化后	下降
TTFT 2,347	152.33	131.68	13.56%
TTFT 65,537	5081.88	4888.70	3.80%
TTFT 131,071	14001.11	13631.31	2.64%
TTFT 253,987	41273.10	40445.43	2.01%
TPOT 2,347	4.661	3.799	18.50%
TPOT 131,071	4.523	4.252	6.00%

四个长度 TTFT ↓2.01%–13.56%
两个已验证长度 TPOT ↓6.00% / 18.50%

离线候选不等于全部接入。端到端对比 CuTeDSL production 路径，仅对应当前服务与验证工况。

AMD 实践：热点算子优化带来模型时延下降

Qwen3.5-4B Decode，AMD MI308X（CDNA3），同一隔离 A/B 工况。

单算子优化

算子耗时：μs

接入算子	优化前	优化后	下降
PA decode	29.7	16.5	**44.4%**
GDN inplace（每层）	17.9	14.2	**20.7%**
LM head	588.9	385.3	**34.6%**

端到端收益

请求响应时间 RT：ms

指标	优化前	优化后	下降
RT	285.5	263.2	**7.8%**

3 次贪心输出指纹一致

轻量校验未发现输出变化，
仍需完整模型精度评测。

AMD 团队提供 FlyDSL 硬件实现支持，AKA 负责优化循环与验证。

收益仅对应当前模型与隔离 A/B 工况。单算子时延下降不能相加为模型收益。

05

如何开始自己的实践

从一个算子的最小闭环起步，再逐步扩展到推理自治。

最小实践：先跑通一个算子的优化闭环

复用开源工具，从一份标准任务开始，再逐步接入自己的服务。

单任务先准备什么

任务 — **Benchmark 与目标卡型**
固定输入域、正确性要求和评分口径

环境 — **Coding Agent 与 GPU 沙箱**
准备所用框架、编译工具链与 Profile 工具

运行 — **预算与独立评测**
先复测基线，再启动迭代并择优

产物 — **Kernel 与可复现记录**
保留候选代码、测量结果与检查点

规模化时再补齐

真实 Trace 与自动任务生成

多卡型资源管理与知识库

框架适配、精度与性能回归

灰度观察与异常回滚

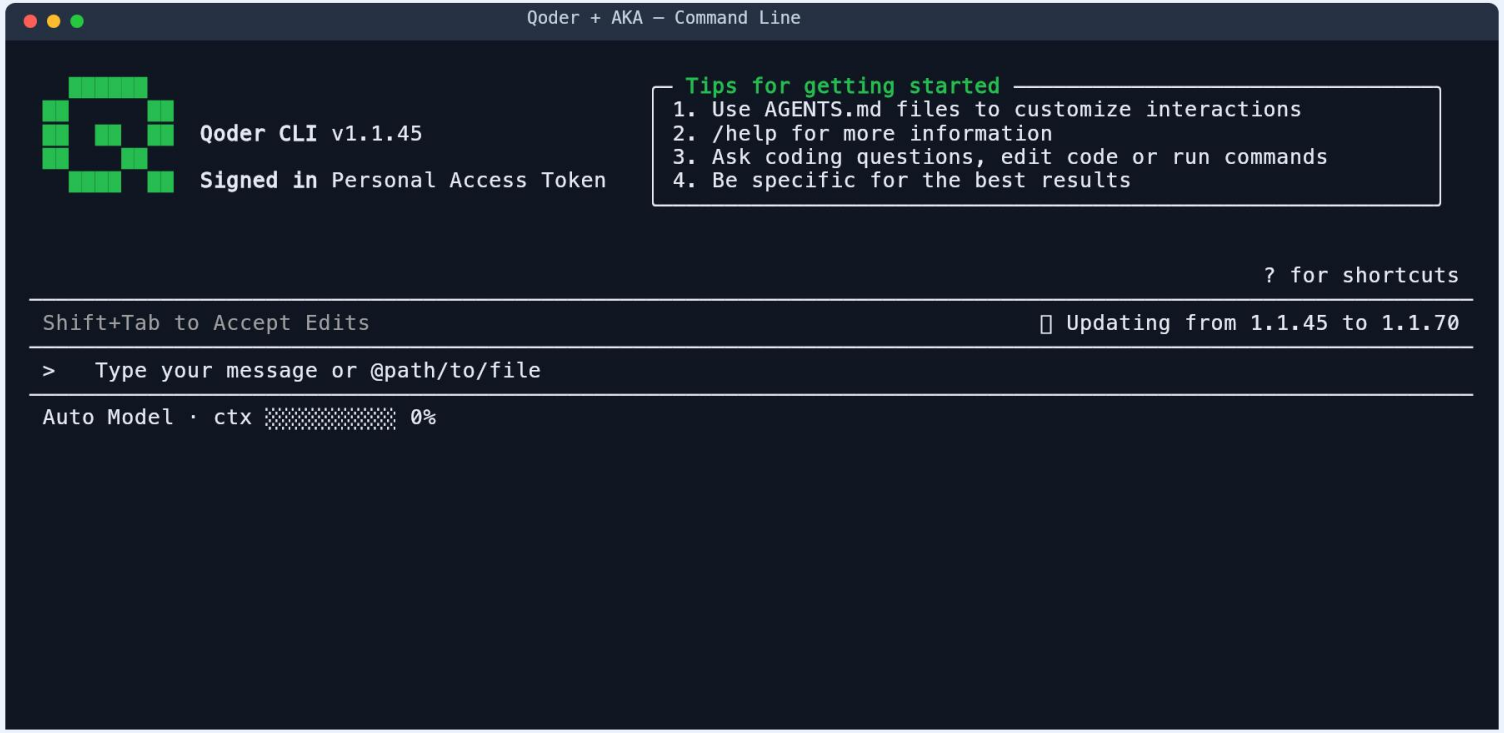
先验证基线能稳定复测，再让 Agent 开始优化。

实操演示：一个算子的自动优化过程

选择任务，启动迭代，查看独立评测与当前最优版本。

ATREX KERNEL AGENT

Qoder 命令行启动演示



前置准备：Qoder 已登录，Benchmark 与 GPU 环境已配置。

RSI 探索：框架与算子优化的端到端闭环

让框架与算子优化共享模型验收，用端到端收益决定哪些改动留下。



Loop 掌握评测与版本推进，算子独立优化，共享服务逐个验收。

精度与主指标由任务定义（TTFT / TPOT / 吞吐等）。
严格模式增加算子收益与 Trace 归因检查，新场景仍需验证适配器。

欢迎关注我们的项目！ Q&A

欢迎 Star、试用与贡献

Atrex-bench

真实工况算子评测

github.com/alibaba/atrex-bench

Atrex-kernel-agent

自动算子优化 Agent

github.com/alibaba/atrex-kernel-agent

Atrex-kernels

高性能算子库

github.com/alibaba/atrex-kernels

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师